

## ✓ Pregunta 2 — Análisis factorial exploratorio

### Métodos Cuantitativos de la Investigación en Negocios — Tarea 2 (2026)

```
1 # Instalación condicional de dependencias para Google Colab.
2 # En ejecución local no instala nada si los paquetes ya están disponibles.
3 import importlib, subprocess, sys
4
5 def instalar_si_falta(import_name, package_name=None):
6     package_name = package_name or import_name
7     try:
8         importlib.import_module(import_name)
9     except ImportError:
10         print(f'Instalando {package_name}...')
11         subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', package_name])
12
13 instalar_si_falta('pyreadstat', 'pyreadstat')
14 instalar_si_falta('factor_analyzer', 'factor-analyzer')
15
16 print('Dependencias verificadas y entorno inicial preparado.')
17
```

Dependencias verificadas y entorno inicial preparado.

## ✓ Enunciado de la pregunta 2

2. La Universidad de Desarrollo está permanentemente tratando de tener los mejores profesores entre sus filas. Como parte de este esfuerzo, la UDD realizó un esfuerzo por analizar el cuestionario basado en la teoría de Blend asociada a los profesores de métodos de investigación. Esta teoría predice que los buenos profesores de métodos cuantitativos poseen cuatro características fundamentales: (1) gran amor por las estadísticas, (2) entusiasmo por el diseño de experimentos, (3) amor por la enseñanza, y (4) una completa ausencia de habilidades interpersonales normales. Estas características pueden estar correlacionadas entre ellas.

El cuestionario “Enseñanza de Experimentos para las Ciencias Sociales” (EECS) ya existe, pero la universidad mejoró este cuestionario y se convirtió en el EECS-R (“Enseñanza de Experimentos para las Ciencias Sociales - Revisado”). Este cuestionario revisado fue enviado y contestado satisfactoriamente por 239 profesores de métodos de investigación alrededor del mundo para evaluar si la teoría de Blend es correcta.

El cuestionario (preguntas, escala y respuestas) se encuentra en el archivo **EECS-R.sav**. Realice un Análisis Factorial Exploratorio y determine si se cumple la teoría de Blend.

## Respuesta

La pregunta se resuelve verificando primero si los datos son adecuados para análisis factorial, determinando cuántos factores retener, interpretando las cargas factoriales y evaluando si la estructura empírica coincide con las cuatro dimensiones propuestas por la teoría de Blend.

```
1 from pathlib import Path
2 from urllib.request import urlretrieve, urlopen
3 import ssl
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 import numpy as np
8 import pandas as pd
9 import pyreadstat
10 from scipy import stats
11 import statsmodels.api as sm
12 import statsmodels.formula.api as smf
13
14 pd.set_option('display.max_columns', 100)
15 pd.set_option('display.width', 160)
16
17 # Base requerida para este notebook.
18 # Funciona localmente y también en Google Colab abierto desde GitHub.
19 REQUIRED_DATA_FILE = 'EECS-R.sav'
20 RAW_DATA_BASE = 'https://raw.githubusercontent.com/sebabecerra/Metodos-Cuantitativos-DEN/main/Tarea2/data'
21
22 def encontrar_o_descargar_base(filename):
23     candidatas = [
24         Path.cwd() / 'data', # ejecución desde Tarea2
25         Path.cwd().parent / 'data', # ejecución desde Tarea2/notebooks
26         Path.cwd() / 'Tarea2' / 'data', # ejecución desde raíz del repo o Colab
27         Path.cwd() # respaldo: base junto al notebook
28     ]
29     for ruta in candidatas:
30         if (ruta / filename).exists():
31             return ruta
32
33 # Si Colab no tiene la carpeta del repo clonada, se descarga la base desde GitHub.
```

```

34 destino_dir = Path.cwd() / 'Tarea2' / 'data'
35 destino_dir.mkdir(parents=True, exist_ok=True)
36 destino = destino_dir / filename
37 url = f'{RAW_DATA_BASE}/{filename}'
38 print(f'Base {filename} no encontrada localmente. Descargando desde GitHub...')
39 try:
40     urlretrieve(url, destino)
41 except Exception as error:
42     # Respaldo útil en algunos entornos locales con problemas de certificados SSL.
43     # En Colab normalmente no se activa, pero evita fallas si Python no encuentra certificados.
44     print('Descarga estándar falló; intentando respaldo SSL...')
45     contexto = ssl._create_unverified_context()
46     with urlopen(url, context=contexto) as respuesta, open(destino, 'wb') as archivo:
47         archivo.write(respuesta.read())
48     return destino_dir
49
50 DATA_DIR = encontrar_o_descargar_base(REQUIRED_DATA_FILE)
51 print('Directorio de datos:', DATA_DIR)
52 print('Base usada:', DATA_DIR / REQUIRED_DATA_FILE)
53

```

Directorio de datos: /Users/sbc/projects/Metodos-Cuantitativos-DEN/Tarea2/data  
Base usada: /Users/sbc/projects/Metodos-Cuantitativos-DEN/Tarea2/data/EECS-R.sav

## 1. Objetivo y decisiones analíticas

Se evalúa si los 28 ítems del EECS-R reproducen cuatro dimensiones correlacionadas propuestas por la teoría de Blend: amor por la estadística, entusiasmo por el diseño experimental, amor por la enseñanza y ausencia de habilidades interpersonales.

Se usa **análisis factorial exploratorio de factores comunes**, extracción de máxima verosimilitud y rotación oblicua Oblimin, porque las dimensiones pueden correlacionarse. Los datos perdidos —solo nueve respuestas— se imputan con la mediana del ítem.

```

1 from factor_analyzer import FactorAnalyzer
2 from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity, calculate_kmo
3 import inspect
4 import sklearn.utils.validation as sk_validation
5 import factor_analyzer.factor_analyzer as fa_module
6
7 # Compatibilidad: algunas versiones nuevas de scikit-learn reemplazan
8 # force_all_finite por ensure_all_finite. factor_analyzer todavía usa
9 # force_all_finite en ciertos entornos, por eso se traduce el argumento.
10 _original_check_array = sk_validation.check_array
11 def _check_array_compat(*args, **kwargs):
12     if (
13         'force_all_finite' in kwargs
14         and 'force_all_finite' not in inspect.signature(_original_check_array).parameters
15     ):
16         kwargs['ensure_all_finite'] = kwargs.pop('force_all_finite')
17     return _original_check_array(*args, **kwargs)
18
19 fa_module.check_array = _check_array_compat
20
21 # Chequeo inicial de la base.
22 # Se carga la base completa; no se filtran observaciones ni se toma una submuestra.
23 df, meta = pyreadstat.read_sav(DATA_DIR/'EECS-R.sav', apply_value_formats=False)
24 items = [f'q{i}' for i in range(1, 29)]
25 X = df[items].copy()
26
27 print('Archivo usado:', DATA_DIR/'EECS-R.sav')
28 print('Número de observaciones:', len(df))
29 print('Número de ítems analizados:', len(items))
30 print('Dimensión de la matriz de ítems:', X.shape)
31 faltantes = int(X.isna().sum().sum())
32 print('Datos perdidos totales en los ítems:', faltantes)
33 print(f'Porcentaje de datos perdidos: {100 * faltantes / X.size:.3f}%')
34 print('\nDatos perdidos por ítem:')
35 print(X.isna().sum().to_string())
36 print('\nDescriptivos básicos de los ítems:')
37 print(X.describe().T[['mean', 'std', 'min', 'max']].round(3).to_string())
38 print('\nEtiquetas de ítems:')
39 for c in items:
40     print(f'{c}: {meta.column_names_to_labels[c]}')
41
42 # Imputación por mediana debido a la baja proporción de datos perdidos.
43 X = X.apply(lambda s: s.fillna(s.median()))
44 print('\nDatos perdidos después de imputar por mediana:', int(X.isna().sum().sum()))
45

```

Archivo usado: /Users/sbc/projects/Metodos-Cuantitativos-DEN/Tarea2/data/EECS-R.sav  
Número de observaciones: 239  
Número de ítems analizados: 28  
Dimensión de la matriz de ítems: (239, 28)  
Datos perdidos totales en los ítems: 9  
Porcentaje de datos perdidos: 0.134%

Datos perdidos por ítem:

|    |   |
|----|---|
| q1 | 0 |
| q2 | 1 |

```

q3      1
q4      0
q5      0
q6      0
q7      0
q8      0
q9      0
q10     0
q11     1
q12     2
q13     0
q14     0
q15     1
q16     0
q17     0
q18     1
q19     1
q20     0
q21     1
q22     0
q23     0
q24     0
q25     0
q26     0
q27     0
q28     0

```

Descriptivos básicos de los ítems:

|     | mean  | std   | min | max |
|-----|-------|-------|-----|-----|
| q1  | 4.720 | 1.412 | 1.0 | 7.0 |
| q2  | 4.782 | 1.391 | 1.0 | 7.0 |
| q3  | 4.929 | 1.158 | 1.0 | 7.0 |
| q4  | 4.791 | 1.243 | 1.0 | 7.0 |
| q5  | 3.406 | 1.315 | 1.0 | 7.0 |
| q6  | 5.276 | 1.387 | 1.0 | 7.0 |
| q7  | 4.987 | 1.102 | 1.0 | 7.0 |
| q8  | 5.213 | 1.277 | 1.0 | 7.0 |
| q9  | 5.314 | 1.249 | 1.0 | 7.0 |
| q10 | 4.444 | 1.454 | 1.0 | 7.0 |
| q11 | 4.080 | 1.545 | 1.0 | 7.0 |
| q12 | 3.865 | 1.423 | 1.0 | 7.0 |
| q13 | 5.431 | 1.304 | 2.0 | 7.0 |
| q14 | 4.895 | 1.575 | 1.0 | 7.0 |
| q15 | 4.639 | 1.773 | 1.0 | 7.0 |
| q16 | 4.619 | 1.757 | 1.0 | 7.0 |
| q17 | 5.456 | 1.256 | 1.0 | 7.0 |
| q18 | 3.958 | 1.411 | 1.0 | 7.0 |

## 2. Factorabilidad: Bartlett y KMO

Bartlett contrasta que la matriz de correlaciones sea identidad. Se requiere  $p < 0.05$ . KMO cuantifica varianza común; valores superiores a 0.80.

La prueba se calcula como  $\chi^2 = -\left(n - 1 - \frac{2p+5}{6}\right) \ln |R|$ , con  $gl = p(p - 1)/2$ , donde  $n$  es el número de casos,  $p$  el número de ítems y  $|R|$  el determinante de la matriz de correlaciones. La regla de decisión es: se rechaza  $H_0$  (matriz identidad) si el estadístico supera el valor crítico  $\chi^2_{0.95, gl}$ , equivalente a  $p < 0.05$ .

```

1 chi,p=calculate_bartlett_sphericity(X); _,kmo=calculate_kmo(X)
2 print(f'Bartlett: chi-cuadrado={chi:.4f}, gl={28*27//2}, ' + ('p<0.001' if p<0.001 else f'p={p:.4g}'))
3 print(f'KMO global={kmo:.4f}')
4
5 # Verificación manual de Bartlett: chi2 = -(n-1-(2p+5)/6)·ln|R|.
6 # Reproducir la fórmula a mano documenta el cálculo y valida la función de la librería.
7 lnR = np.linalg.slogdet(X.corr().to_numpy())[1]
8 chi_manual = -(len(X)-1-(2*28+5)/6)*lnR
9 print(f'\nln|R| = {lnR:.4f}')
10 print(f'chi2 manual = -(239-1-61/6)·({lnR:.4f}) = {chi_manual:.4f} (coincide con la función)')
11 print(f'chi2 crítico (95%, gl=378) = {stats.chi2.ppf(.95,378):.4f} -> como {chi:.2f} > {stats.chi2.ppf(.95,378):.2f}, se rechaza H0')

```

Bartlett: chi-cuadrado=3045.3894, gl=378, p<0.001  
KMO global=0.8952

ln|R| = -13.3667  
chi2 manual = -(239-1-61/6)·(-13.3667) = 3045.3894 (coincide con la función)  
chi2 crítico (95%, gl=378) = 424.3342 -> como 3045.39 > 424.33, se rechaza H0

## 3. Número de factores

Para decidir cuántos factores retener se combinan tres criterios:

1. **Autovalores observados:** indican cuánta varianza explica cada dimensión inicial.
2. **Gráfico de sedimentación:** muestra los autovalores ordenados de mayor a menor; se observa el punto donde la curva comienza a aplanarse, porque desde ahí los factores adicionales aportan poca información nueva.
3. **Análisis paralelo:** compara los autovalores observados con los que aparecerían por azar en matrices aleatorias del mismo tamaño.

El análisis paralelo es especialmente importante porque el criterio de autovvalor mayor que 1 puede sugerir más factores de los realmente respaldados por los datos. En esta aplicación, el cuarto factor queda muy cerca del percentil 95 aleatorio. El valor 1.501 corresponde al

percentil 95 del cuarto autovalor obtenido en las 1.000 matrices aleatorias generadas para el análisis paralelo. Por eso se revisa también la media de autovalores aleatorios y la coherencia teórica de cuatro dimensiones.

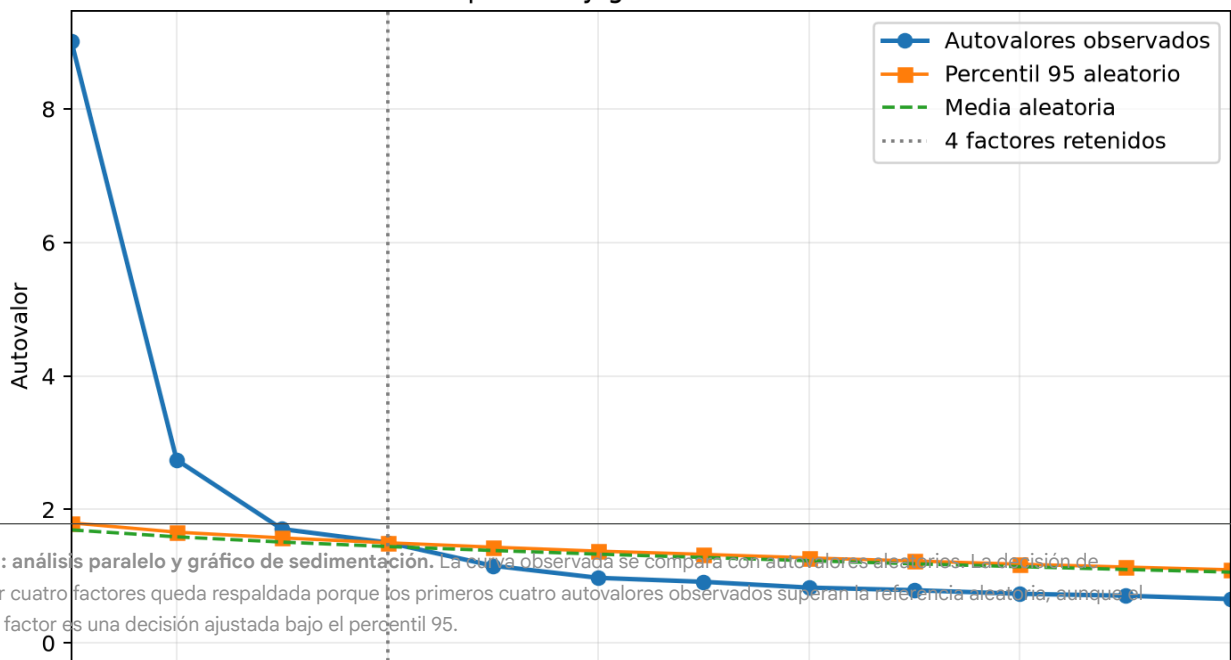
```

1 R=X.corr().to_numpy(); eig=np.linalg.eigvalsh(R)[::-1]
2 rng=np.random.default_rng(12345)
3 rand=[]
4 for _ in range(1000):
5     Rr=np.corrcoef(rng.normal(size=X.shape),rowvar=False)
6     rand.append(np.linalg.eigvalsh(Rr)[::-1])
7 rand=np.array(rand)
8 rand95=np.percentile(rand,95,axis=0)
9 randmean=rand.mean(axis=0)
10 print('Primeros 8 autovalores observados:',np.round(eig[:8],3))
11 print('Percentil 95 aleatorio:',np.round(rand95[:8],3))
12 print('Media aleatoria:',np.round(randmean[:8],3))
13 print('Factores retenidos por percentil 95:',int((eig>rand95).sum()))
14 print('Factores retenidos por media aleatoria:',int((eig>randmean).sum()))
15 print(f'Cuarto autovalor observado = {eig[3]:.3f}; percentil 95 aleatorio = {rand95[3]:.3f}; media aleatoria = {randmean[3]:.3f}')
16
17 import matplotlib.pyplot as plt
18 from pathlib import Path
19 from IPython.display import Image, display
20 ASSETS_DIR = Path('../assets')
21 ASSETS_DIR.mkdir(parents=True, exist_ok=True)
22 fig, ax = plt.subplots(figsize=(8,5))
23 xs=np.arange(1,len(items)+1)
24 ax.plot(xs,eig,marker='o',label='Autovalores observados',linewidth=2)
25 ax.plot(xs,rand95,marker='s',label='Percentil 95 aleatorio',linewidth=1.5)
26 ax.plot(xs,randmean,linestyle='--',label='Media aleatoria',linewidth=1.5)
27 ax.axvline(4,color='gray',linestyle=':',label='4 factores retenidos')
28 ax.set_xlabel('Número de factor')
29 ax.set_ylabel('Autovalor')
30 ax.set_title('Análisis paralelo y gráfico de sedimentación')
31 ax.set_xlim(1,12)
32 ax.grid(alpha=.25)
33 ax.legend()
34 fig.savefig(ASSETS_DIR / 'p2_scree_parallel.png', dpi=200, bbox_inches='tight')
35 plt.close(fig)
36 display(Image(str(ASSETS_DIR / 'p2_scree_parallel.png'))))
37 print(f'Figura guardada: {ASSETS_DIR / 'p2_scree_parallel.png'}')

```

Primeros 8 autovalores observados: [9.03 2.725 1.706 1.505 1.157 0.975 0.917 0.831]  
 Percentil 95 aleatorio: [1.785 1.656 1.55 1.501 1.437 1.367 1.312 1.268]  
 Media aleatoria: [1.687 1.567 1.494 1.445 1.388 1.333 1.285 1.239]  
 Factores retenidos por percentil 95: 4  
 Factores retenidos por media aleatoria: 4  
 Cuarto autovalor observado = 1.505; percentil 95 aleatorio = 1.501; media aleatoria = 1.445  
 Figura guardada: ../assets/p2\_scree\_parallel.png

### Análisis paralelo y gráfico de sedimentación



**Figura: análisis paralelo y gráfico de sedimentación.** La cuarta observada se compara con autovalores aleatorios. La decisión de retener cuatro factores queda respaldada porque los primeros cuatro autovalores observados superan la referencia aleatoria, aunque el cuarto factor es una decisión ajustada bajo el percentil 95.

```

1 fa=FactorAnalyzer(n_factors=4,rotation='oblimin',method='ml'); fa.fit(X)
2 L=pd.DataFrame(fa.loadings_,index=items,columns=['F1','F2','F3','F4'])
3 print(L.round(3).to_string())
4 print('\nVarianza por factor:')
5 varianza = pd.DataFrame(np.array(fa.get_factor_variance()).T,columns=['SS cargas','Proporción','Acumulada'],index=['F1',
6 print(varianza.round(4).to_string())
7 print('\nCorrelaciones factoriales:')
8 print(pd.DataFrame(fa.phi_,index=L.columns,columns=L.columns).round(3).to_string())

```

|     | F1     | F2     | F3     | F4     |
|-----|--------|--------|--------|--------|
| q1  | 0.401  | -0.140 | 0.285  | 0.340  |
| q2  | 0.097  | -0.329 | 0.357  | 0.272  |
| q3  | 0.116  | 0.293  | -0.048 | 0.508  |
| q4  | 0.125  | 0.165  | 0.137  | 0.510  |
| q5  | 0.056  | 0.055  | 0.669  | -0.131 |
| q6  | 0.160  | -0.347 | 0.161  | 0.264  |
| q7  | 0.139  | 0.452  | -0.049 | 0.299  |
| q8  | 0.575  | 0.222  | 0.036  | 0.242  |
| q9  | 0.437  | -0.114 | 0.258  | 0.372  |
| q10 | 0.417  | -0.069 | 0.178  | 0.044  |
| q11 | 0.266  | 0.457  | 0.200  | 0.102  |
| q12 | -0.011 | 0.133  | 0.269  | -0.164 |
| q13 | 0.560  | 0.082  | -0.033 | 0.209  |
| q14 | 0.757  | 0.028  | 0.179  | -0.337 |
| q15 | 0.361  | -0.017 | 0.137  | 0.184  |
| q16 | 0.741  | 0.215  | -0.024 | -0.200 |
| q17 | 0.788  | -0.009 | -0.034 | 0.136  |
| q18 | 0.232  | -0.322 | 0.404  | 0.142  |
| q19 | -0.055 | 0.568  | -0.177 | 0.078  |
| q20 | 0.035  | 0.450  | 0.052  | 0.199  |
| q21 | 0.471  | 0.165  | -0.040 | 0.270  |
| q22 | 0.857  | -0.074 | -0.021 | 0.113  |
| q23 | -0.094 | 0.042  | 0.700  | 0.066  |
| q24 | 0.097  | 0.284  | 0.127  | 0.481  |
| q25 | 0.101  | 0.533  | 0.131  | 0.082  |
| q26 | 0.278  | 0.593  | 0.020  | -0.072 |
| q27 | -0.020 | 0.612  | 0.382  | 0.068  |
| q28 | 0.012  | 0.211  | 0.533  | -0.016 |

Varianza por factor:

|    | SS cargas | Proporción | Acumulada |
|----|-----------|------------|-----------|
| F1 | 4.3219    | 0.1544     | 0.1544    |
| F2 | 2.7152    | 0.0970     | 0.2513    |
| F3 | 2.1239    | 0.0759     | 0.3272    |
| F4 | 1.7733    | 0.0633     | 0.3905    |

Correlaciones factoriales:

|    | F1    | F2    | F3    | F4    |
|----|-------|-------|-------|-------|
| F1 | 1.000 | 0.299 | 0.471 | 0.370 |
| F2 | 0.299 | 1.000 | 0.130 | 0.129 |
| F3 | 0.471 | 0.130 | 1.000 | 0.252 |
| F4 | 0.370 | 0.129 | 0.252 | 1.000 |

#### ✓ 4. Método de extracción: máxima verosimilitud y contraste con método principal

En el análisis factorial exploratorio existen distintas formas de extraer factores. En este notebook se reportan dos aproximaciones para que la decisión metodológica sea transparente.

Primero, se estima la solución principal mediante **máxima verosimilitud (ML)**. Esta especificación es la utilizada como referencia porque el objetivo de la pregunta es evaluar dimensiones latentes de la teoría de Blend. ML busca modelar la varianza común atribuible a factores latentes y separar esa varianza del error de medición.

Segundo, se estima una solución de sensibilidad mediante el método **principal**. Esta alternativa suele reportar una varianza acumulada más alta porque reproduce una mayor proporción de la varianza observada de los ítems. Por eso es útil como comparación, pero no se adopta como especificación principal: explicar más varianza observada no implica necesariamente representar mejor constructos latentes.

Ambas soluciones se estiman con rotación Oblimin, porque el enunciado indica que las dimensiones pueden estar correlacionadas.

```

1 # Comparación entre extracción por máxima verosimilitud y método principal.
2 # Se usa la misma base imputada y la misma rotación Oblimin para aislar el efecto del método de extracción.
3
4 fa_ml = FactorAnalyzer(n_factors=4, rotation='oblimin', method='ml')
5 fa_ml.fit(X)
6 var_ml = pd.DataFrame(
7     np.array(fa_ml.get_factor_variance()).T,
8     columns=['SS cargas', 'Proporción', 'Acumulada'],
9     index=['F1', 'F2', 'F3', 'F4'])
10
11
12 fa_principal = FactorAnalyzer(n_factors=4, rotation='oblimin', method='principal')
13 fa_principal.fit(X)
14 var_principal = pd.DataFrame(
15     np.array(fa_principal.get_factor_variance()).T,
16     columns=['SS cargas', 'Proporción', 'Acumulada'],
17     index=['F1', 'F2', 'F3', 'F4'])
18
19
20 comparacion_metodos = pd.DataFrame({
21     'Método': ['Máxima verosimilitud (ML)', 'Principal'],
22     'Rotación': ['Oblimin', 'Oblimin'],
23     'n': [len(X), len(X)],
24     'Varianza acumulada 4 factores': [var_ml['Acumulada'].iloc[-1], var_principal['Acumulada'].iloc[-1]]
25 })
26
27 print('Comparación de métodos de extracción con la misma muestra imputada:')
28 print(comparacion_metodos.assign(**{'Varianza acumulada 4 factores': comparacion_metodos['Varianza acumulada 4 factores']
29 print('

```

```

30 Varianza por factor - ML:')
31 print(var_ml.round(4).to_string())
32 print('
33 Varianza por factor - Principal:')
34 print(var_principal.round(4).to_string())
35 print('
36 Lectura: Principal explica más varianza observada, pero ML se mantiene como referencia porque modela factores latentes c
37

```

Comparación de métodos de extracción con la misma muestra imputada:

|                           | Método Rotación | n       | Varianza acumulada 4 factores |        |
|---------------------------|-----------------|---------|-------------------------------|--------|
| Máxima verosimilitud (ML) | Oblimin         | 239     | 39.05%                        |        |
|                           | Principal       | Oblimin | 239                           | 47.82% |

Varianza por factor - ML:

|    | SS cargas | Proporción | Acumulada |
|----|-----------|------------|-----------|
| F1 | 4.3219    | 0.1544     | 0.1544    |
| F2 | 2.7152    | 0.0970     | 0.2513    |
| F3 | 2.1239    | 0.0759     | 0.3272    |
| F4 | 1.7733    | 0.0633     | 0.3905    |

Varianza por factor - Principal:

|    | SS cargas | Proporción | Acumulada |
|----|-----------|------------|-----------|
| F1 | 4.8913    | 0.1747     | 0.1747    |
| F2 | 2.4978    | 0.0892     | 0.2639    |
| F3 | 2.4107    | 0.0861     | 0.3500    |
| F4 | 3.5892    | 0.1282     | 0.4782    |

Lectura: Principal explica más varianza observada, pero ML se mantiene como referencia porque modela factores latentes comunes

## 5. Interpretación y confiabilidad

La solución se interpreta considerando conjuntamente contenido y cargas:

- F1: entusiasmo por diseño experimental.
- F2: amor por la enseñanza.
- F3: ausencia de habilidades interpersonales.
- F4: amor por la estadística.

Se calcula alfa con conjuntos conceptualmente coherentes. Algunos ítems presentan cargas cruzadas o débiles, por lo que la teoría se evalúa en grado, no solo contando factores.

```

1 def alpha(d):
2     k=d.shape[1]; return k/(k-1)*(1-d.var(ddof=1).sum()/d.sum(axis=1).var(ddof=1))
3 sets={
4     'Diseño experimental':['q8','q13','q14','q16','q17','q22'],
5     'Enseñanza':['q7','q10','q11','q19','q20','q25','q26'],
6     'Habilidades interpersonales deficientes':['q2','q5','q6','q12','q18','q23','q27','q28'],
7     'Estadística':['q1','q3','q4','q9','q15','q21','q24']}
8 for nombre,its in sets.items(): print(f'{nombre}: alpha={alpha(X[its]):.3f} ({len(its)} ítems)')

```

Diseño experimental: alpha=0.880 (6 ítems)  
 Enseñanza: alpha=0.747 (7 ítems)  
 Habilidades interpersonales deficientes: alpha=0.691 (8 ítems)  
 Estadística: alpha=0.830 (7 ítems)

```

1 # Correlaciones ítem-total corregidas por subescala teórica.
2 # Verifica que ningún ítem esté invertido o mal codificado dentro de su subescala:
3 # una correlación ítem-total negativa indicaría necesidad de recodificación antes del alfa.
4 for nombre, its in sets.items():
5     sub = X[its]
6     print(f'\n{n{nombre}:}')
7     for it in its:
8         resto = sub.drop(columns=it).sum(axis=1)
9         r = sub[it].corr(resto)
10        marca = ' <-- NEGATIVA' if r < 0 else ''
11        print(f' {it}: r ítem-total corregida = {r:.3f}{marca}')

```

Diseño experimental:

q8: r ítem-total corregida = 0.714  
 q13: r ítem-total corregida = 0.611  
 q14: r ítem-total corregida = 0.634  
 q16: r ítem-total corregida = 0.678  
 q17: r ítem-total corregida = 0.767  
 q22: r ítem-total corregida = 0.783

Enseñanza:

q7: r ítem-total corregida = 0.476  
 q10: r ítem-total corregida = 0.248  
 q11: r ítem-total corregida = 0.538  
 q19: r ítem-total corregida = 0.360  
 q20: r ítem-total corregida = 0.505  
 q25: r ítem-total corregida = 0.568  
 q26: r ítem-total corregida = 0.583

Habilidades interpersonales deficientes:

q2: r ítem-total corregida = 0.382  
 q5: r ítem-total corregida = 0.486

```

q6: r item-total corregida = 0.261
q12: r item-total corregida = 0.170
q18: r item-total corregida = 0.446
q23: r item-total corregida = 0.566
q27: r item-total corregida = 0.333
q28: r item-total corregida = 0.424

```

Estadística:

```

q1: r item-total corregida = 0.636
q3: r item-total corregida = 0.516
q4: r item-total corregida = 0.539
q9: r item-total corregida = 0.626
q15: r item-total corregida = 0.529
q21: r item-total corregida = 0.603
q24: r item-total corregida = 0.647

```

## 6. Conclusión

Bartlett es significativo ( $\chi^2(378) = 3045.39$ ,  $p < 0.001$ ) y KMO=0.895, por lo que la matriz es adecuada. El análisis paralelo recomienda **cuatro factores** tanto con el percentil 95 (cuarto autovalor 1.505 contra umbral 1.501, decisión ajustada) como con la media de los autovalores aleatorios (umbral 1.445, decisión holgada), coincidiendo con Blend y con el respaldo de la expectativa teórica a priori. La solución rotada permite reconocer las cuatro áreas teóricas, especialmente diseño experimental y estadística/enseñanza.

Sin embargo, hay cargas cruzadas (por ejemplo q1, q9 y q27) e ítems sin carga factorial relevante o débiles (como q12), y la dimensión de habilidades interpersonales deficientes no queda completamente limpia. **La teoría de Blend recibe apoyo general respecto del número y contenido de factores, pero no apoyo perfecto a nivel de todos los ítems.** Se recomienda revisar o eliminar ítems ambiguos antes de una validación confirmatoria.

## ✓ Apéndice A. Especificaciones alternativas de sensibilidad

Este apéndice reproduce dos variantes útiles para entender cómo cambian algunos resultados bajo decisiones alternativas. No se usan como especificación principal.

1. **Principal + Oblimin + listwise, sin (ID)**: reproduce la lógica de una especificación con eliminación de casos incompletos. Esta variante alcanza una varianza acumulada de 47.88%, muy cercana a la solución principal con imputación y método principal. La diferencia principal no proviene de los casos faltantes, sino del método de extracción.
2. **Minres + Varimax + listwise, sin (ID)**: permite ver qué ocurre cuando se impone una rotación ortogonal. No se prefiere porque Varimax fuerza factores no correlacionados, mientras que el enunciado permite que las dimensiones estén correlacionadas.

En ambos casos se excluye (ID), porque no es un ítem del EECS-R ni mide un rasgo latente.

```

1 # --- Apéndice A: especificaciones alternativas sin ID ---
2 df_sens, _ = pyreadstat.read_sav(DATA_DIR / REQUIRED_DATA_FILE, apply_value_formats=False)
3 X_listwise = df_sens[items].dropna()
4
5 # A1. Principal + Oblimin + listwise
6 fa_principal_listwise = FactorAnalyzer(n_factors=4, rotation='oblimin', method='principal')
7 fa_principal_listwise.fit(X_listwise)
8 var_principal_listwise = pd.DataFrame(
9     np.array(fa_principal_listwise.get_factor_variance()).T,
10     columns=['SS cargas', 'Proporción', 'Acumulada'],
11     index=['F1', 'F2', 'F3', 'F4'])
12 )
13
14 # A2. Minres + Varimax + listwise
15 fa_varimax = FactorAnalyzer(n_factors=4, rotation='varimax', method='minres')
16 fa_varimax.fit(X_listwise)
17 var_varimax = pd.DataFrame(
18     np.array(fa_varimax.get_factor_variance()).T,
19     columns=['SS cargas', 'Proporción', 'Acumulada'],
20     index=['F1', 'F2', 'F3', 'F4'])
21 )
22 L_varimax = pd.DataFrame(fa_varimax.loadings_, index=items, columns=['F1', 'F2', 'F3', 'F4'])
23
24 print('Apéndice de sensibilidad sin ID')
25 print(f'Observaciones listwise usadas: {len(X_listwise)}')
26 print(f'Observaciones descartadas por listwise: {len(df_sens) - len(X_listwise)}')
27 print('
28 A1. Principal + Oblimin + listwise')
29 print(var_principal_listwise.round(4).to_string())
30 print(f'Varianza acumulada A1: {var_principal_listwise["Acumulada"].iloc[-1]*100:.2f}%')
31 print('
32 A2. Minres + Varimax + listwise')
33 print(var_varimax.round(4).to_string())
34 print(f'Varianza acumulada A2: {var_varimax["Acumulada"].iloc[-1]*100:.2f}%')
35 print('
36 Cargas factoriales Varimax:')
37 print(L_varimax.round(3).to_string())
38 print('
39 Lectura: estas alternativas son útiles para sensibilidad. La especificación principal sigue siendo ML + Oblimin con n=23
40

```

Apéndice de sensibilidad sin ID  
Observaciones listwise usadas: 231

Observaciones descartadas por listwise: 8

A1. Principal + Oblimin + listwise  
SS cargas Proporción Acumulada  
F1 4.8080 0.1717 0.1717  
F2 2.5837 0.0923 0.2640  
F3 2.3110 0.0825 0.3465  
F4 3.7041 0.1323 0.4788  
Varianza acumulada A1: 47.88%

A2. Minres + Varimax + listwise  
SS cargas Proporción Acumulada  
F1 4.1833 0.1494 0.1494  
F2 3.8139 0.1362 0.2856  
F3 3.2596 0.1164 0.4020  
F4 1.6889 0.0603 0.4623  
Varianza acumulada A2: 46.23%

Cargas factoriales Varimax:

|     | F1     | F2    | F3     | F4     |
|-----|--------|-------|--------|--------|
| q1  | 0.242  | 0.358 | 0.612  | 0.108  |
| q2  | -0.018 | 0.092 | 0.614  | 0.111  |
| q3  | 0.638  | 0.149 | 0.235  | -0.028 |
| q4  | 0.538  | 0.098 | 0.435  | 0.021  |
| q5  | 0.034  | 0.201 | 0.294  | 0.542  |
| q6  | -0.038 | 0.098 | 0.514  | 0.008  |
| q7  | 0.602  | 0.207 | 0.044  | 0.027  |
| q8  | 0.501  | 0.544 | 0.310  | 0.103  |
| q9  | 0.292  | 0.385 | 0.608  | 0.103  |
| q10 | 0.106  | 0.360 | 0.345  | 0.129  |
| q11 | 0.530  | 0.347 | 0.106  | 0.301  |
| q12 | 0.043  | 0.035 | -0.031 | 0.320  |
| q13 | 0.352  | 0.519 | 0.276  | 0.020  |
| q14 | 0.039  | 0.701 | 0.139  | 0.343  |
| q15 | 0.223  | 0.366 | 0.341  | 0.082  |
| q16 | 0.253  | 0.709 | 0.020  | 0.192  |
| q17 | 0.300  | 0.671 | 0.364  | 0.049  |
| q18 | -0.055 | 0.193 | 0.551  | 0.240  |
| q19 | 0.481  | 0.024 | -0.283 | 0.032  |
| q20 | 0.514  | 0.136 | 0.009  | 0.150  |
| q21 | 0.462  | 0.451 | 0.240  | -0.004 |
| q22 | 0.260  | 0.722 | 0.419  | 0.045  |
| q23 | 0.114  | 0.027 | 0.400  | 0.526  |
| q24 | 0.595  | 0.138 | 0.322  | 0.045  |
| q25 | 0.549  | 0.182 | -0.059 | 0.261  |
| q26 | 0.528  | 0.370 | -0.204 | 0.275  |
| q27 | 0.575  | 0.155 | 0.004  | 0.471  |
| q28 | 0.215  | 0.123 | 0.207  | 0.496  |

Lectura: estas alternativas son útiles para sensibilidad. La especificación principal sigue siendo ML + Oblimin con n=239.